conosci modelli open che supportano 4 gb di ram o vram che sappiano programmare e conoscono l'italiano ?

Lavorato per 15s >

Sì, ci sono diversi modelli open-source (o open-weight) che girano su ~4 GB di VRAM/RAM, quantizzati in Q4_K_M (o simili), capaci di programmare decentemente e con supporto multilingue che include l'italiano.

I migliori per te (priorità coding + italiano + 4 GB)

Modello	Dimensione tipica (Q4)	Perché è adatto	Note
Qwen2.5-Coder 3B (o 1.5B)	~1.9 GB (3B) / ~1 GB (1.5B)	Specializzato in coding, molto forte per le dimensioni. Supporto multilingue ottimo (italiano compreso).	**Scelta n.1 per programmazione** su 4 GB. Contesto economico, veloce.
Qwen2.5 3B (o Qwen3.5 / Qwen3 2B-4B)	~2–2.5 GB	All-rounder eccellente, coding solido + multilingue forte (29+ lingue, italiano molto buono).	Ottimo equilibrio. La famiglia Qwen è tra le più citate per italiano.
Phi-4 Mini (3.8B)	~2.3–2.5 GB	Ragionamento e coding sorprendentemente buoni per la taglia.	Molto consigliato per 4 GB; meno "specializzato codice" di Qwen-Coder ma più versatile.
Llama 3.2 3B	~2 GB	Buono in generale, ecosistema ampio, multilingue discreto.	Solido e facile da usare.
Gemma 3 / Gemma 4 E2B (o 4B)	~1.3–2.5 GB	Buona qualità, multilingue (italiano ok), a volte vision.	Google, performante per le dimensioni.
StarCoder2 3B	~1.8–2 GB	Focalizzato sul codice (completamento, FIM).	Meno forte in chat/italiano rispetto a Qwen.
Granite 4.x 3B (IBM)	~1.8–2.1 GB	Buoni risultati coding pubblicati, modalità di "thinking".	Alternativa interessante e leggera.

Raccomandazioni pratiche

- **Per coding prioritario**: inizia con `qwen2.5-coder:3b` (Ollama) o la versione 1.5B se vuoi più margine. È il più citato come miglior modello di codice che entra comodamente in 4 GB.
- **Per bilanciare italiano + coding + uso generale**: **Qwen2.5 3B** o le versioni piccole della serie Qwen3/Qwen3.5.
- **Phi-4 Mini** se vuoi ragionamento più forte.

Come farli girare

- **Ollama** (più semplice):
  ```
  ollama pull qwen2.5-coder:3b
  ollama run qwen2.5-coder:3b
  ```
- Alternativa: **LM Studio** (interfaccia grafica comoda).

- Usa quantizzazione **Q4_K_M** (o Q4_0 / IQ4). Lascia un po' di margine per il contesto (KV cache); con 4 GB totali è meglio non spingere contesti lunghissimi.

Cosa aspettarti

Su 4 GB questi modelli gestiscono bene:

- Generazione di funzioni/script
- Spiegazione e debugging di codice
- Completamento e refactoring leggero
- Conversazioni in italiano (soprattutto Qwen)

Non aspettarti prestazioni da modelli 14B–32B o da cloud frontier: sono "piccoli ma utili". Per progetti complessi o codice molto lungo la qualità scende.

I modelli della famiglia **Qwen** sono quelli che attualmente danno il miglior rapporto qualità/coding/italiano a queste dimensioni. Se mi dici se usi Ollama, LM Studio, o che linguaggio programmi di più, posso restringere ulteriormente.

bla bla x:~$ ollama list
NAME ID SIZE MODIFIED
qwen3.5:latest 56671c2ab938 6.6 GB 23 hours ago
qwen3.5-16k:latest 834455170209 3.3 GB 23 hours ago
gemma4:31b-cloud ef09f235533c - 34 hours ago
qwen3.5:4b d8b0f5e9760c 3.3 GB 7 days ago
qwen3:1.7b 8f68893c685c 1.4 GB 7 days ago
deepseek-r1:1.5b e0979632db5a 1.1 GB 7 days ago
qwen2.5-coder:3b f72c60cabf62 1.9 GB 7 days ago
qwen2.5-coder:1.5b d7372fd82851 986 MB 7 days ago
llama2-uncensored:latest 44040b922233 3.8 GB 2 weeks ago
tev1:4b 9b5bb969e46c 4.5 GB 2 weeks ago
qwen3:8b 500a1f067a9f 5.2 GB 3 weeks ago
translategemma:4b c49d986b0764 3.3 GB 2 months ago
qwen2.5:7b 845dbda0ea48 4.7 GB 3 months ago
qwen2.5vl:3b fb90415cde1e 3.2 GB 3 months ago
antconsales/antonio-gemma3-evo-q4:latest 26d5dd4a1998 720 MB 3 months ago
moondream:latest 55fc3abd3867 1.7 GB 3 months ago
llama3.2:latest a80c4f17acd5 2.0 GB 3 months ago
nomic-embed-text:latest 0a109f422b47 274 MB 3 months ago
hf.co/hwpoison89/gemma-1B-LinuxCLI-GGUF:BF16 b9cfe4a9f474 2.0 GB 3 months ago
mistral:7b-instruct 6577803aa9a0 4.4 GB 3 months ago
deepseek-coder:6.7b ce298d984115 3.8 GB 3 months ago
phi3:3.8b 4f2222927938 2.2 GB 3 months ago
deepseek-v4-flash:cloud ea027821675c - 3 months ago
gemini-3-flash-preview:latest ebade0d31690 - 3 months ago
qwen3:4b 359d7dd4bcda 2.5 GB 3 months ago
granite4:1b 26e4cb132798 3.3 GB 3 months ago
deepseek-coder:6.7b-instruct-q4_K_M af6da0444f84 4.1 GB 3 months

```
ago
llama3.2:3b                    a80c4f17acd5    2.0 GB    3 months ago
embeddinggemma:latest          85462619ee72    621 MB    3 months ago
qwen3-next:80b-cloud           f5ccd68d2872    -         4 months ago
wildlux@wildlux-hp15cb0xx:~$
```

Lavorato per 14s >

Hai già installato **quasi tutti i migliori** per 4 GB + coding + italiano.

I tuoi modelli più adatti (ordinati per utilità)

Modello	Dimensione	Uso consigliato	Commento
qwen2.5-coder:3b	1.9 GB	**Coding principale**	Il più forte per programmare tra quelli che hai. Usalo come default.
qwen2.5-coder:1.5b	986 MB	Coding veloce / autocomplete	Quando vuoi massima velocità o più contesto libero.
qwen3:4b / **qwen3.5:4b**	2.5–3.3 GB	Generale + coding + italiano	Ottimi all-rounder, italiano molto buono.
qwen3:1.7b	1.4 GB	Uso leggero / veloce	Quando la VRAM è stretta.
phi3:3.8b	2.2 GB	Ragionamento + coding	Alternativa solida a Qwen.
llama3.2:3b	2.0 GB	Generale	Buono ma un po' inferiore a Qwen in coding e italiano.
deepseek-coder:6.7b	3.8–4.1 GB	Coding (se entra)	Più potente, ma stretto su 4 GB (può andare in offload).

Cosa ti consiglio di usare adesso

1. **Per programmare** → `qwen2.5-coder:3b`

   ```bash
   ollama run qwen2.5-coder:3b
   ```

2. **Per chat in italiano + un po' di codice** → `qwen3.5:4b` o `qwen3:4b`
3. **Se la VRAM è davvero al limite** → `qwen2.5-coder:1.5b` o `qwen3:1.7b`

Consigli pratici

- Con 4 GB di VRAM lascia sempre un po' di margine (non caricare modelli da 3.8–4.1 GB con contesti lunghi).
- I modelli `qwen3.5:latest` (6.6 GB) e quelli cloud non servono per l'uso locale su 4 GB.
- Puoi anche creare un alias o un Modelfile se vuoi personalizzare temperatura / system prompt.

Vuoi che ti dia i comandi per testarli in parallelo o per creare un modello personalizzato (es. system prompt in italiano + coding)?

quindi se metto un 4-5-7 bilioni di parametri posso aumentare il contesto e avere più performance quindi

Lavorato per 18s >

Sì e no — dipende da come lo usi.

La regola base su 4 GB di VRAM

Parametri	Dimensione tipica (Q4)	Contesto utilizzabile	Performance	Commento
1.5–3B	1–2 GB	Alto (8k–16k+ comodo)	Buona	Miglior equilibrio su 4 GB
4B	2.5–3.3 GB	Medio (4k–8k)	Migliore	Ancora gestibile
7B	4.0–4.7 GB	Basso (2k–4k) o offload	Ancora migliore	Stretto, spesso rallenta

Cosa succede realmente

1. **Più parametri = più qualità**
   Un 7B è generalmente più intelligente di un 3B (migliore ragionamento, codice più corretto, italiano più naturale).
2. **Ma su 4 GB di VRAM**
   - Un modello da 7B occupa quasi tutta la memoria → **rimane poco spazio per il contesto** (KV cache).
   - Se forzi un contesto lungo, Ollama/llama.cpp scarica parti sulla RAM/CPU → **diventa molto più lento**.
3. **Quindi il trade-off è questo**:
   - Vuoi **massima qualità** e accetti contesto corto + possibile rallentamento → prova un 7B (es. `qwen2.5:7b` o `deepseek-coder:6.7b` che hai già).
   - Vuoi **velocità + contesto decente** → resta sui 3B–4B (soprattutto `qwen2.5-coder:3b` e `qwen3:4b`).

Consiglio pratico per te

- **Coding quotidiano** → continua con `qwen2.5-coder:3b` (è già molto forte e lascia spazio).
- **Quando ti serve più "cervello"** → prova `qwen3:4b` o `qwen3.5:4b` (hai già entrambi).
- **7B** provalo solo se ti serve davvero più qualità e sei disposto a tenere il contesto basso (o ad accettare un po' di offload sulla CPU).

Vuoi che ti dica esattamente quali dei modelli che hai già installato sono più convenienti da usare in base a contesto lungo vs qualità massima?

Cosa significa offload sulla CPU?

Lavorato per 12s >

Offload sulla CPU significa che una parte del modello (o del contesto) non sta più sulla scheda video (GPU/VRAM), ma viene spostata sulla memoria di sistema (RAM) e viene calcolata dal processore (CPU).

Come funziona in pratica

1. **Ideale (tutto su GPU)**
   Tutto il modello + il contesto sta nella VRAM → il calcolo è veloce (decine di token al secondo).
2. **Quando la VRAM non basta**
   Ollama/llama.cpp dice: “non ci sta tutto”, allora:
   - Tiene le parti più importanti sulla GPU
   - **Sposta (offload) il resto sulla RAM/CPU**
3. **Cosa succede quando c’è offload**
   - La velocità cala **molto** (spesso da 30-50 token/s a 5-15 token/s o anche meno)
   - Il PC usa di più la RAM e la CPU
   - A volte diventa “a scatti” o più lento del previsto

Esempio concreto con i tuoi modelli

- `qwen2.5-coder:3b` (1.9 GB) → di solito **tutto in GPU**, veloce
- `deepseek-coder:6.7b` o `qwen2.5:7b` (≈4 GB) → su 4 GB di VRAM spesso **parte in offload** → più lento

Come capire se sta facendo offload

Quando lanci il modello puoi controllare con:

```Bash
ollama ps
```

Se vedi qualcosa tipo `PROCESSOR` o se la VRAM non è usata al 100% e la velocità è bassa, sta facendo offload.

In sintesi: **offload = il modello è troppo grande per la tua VRAM e parte del lavoro viene scaricato sulla CPU**, rendendolo più lento.

ok lo rende più lento ma con qualità maggiore

Lavorato per 6s >

Esatto.

Riassumendo il trade-off:

- **Modello più grande (es. 7B)** →
  qualità più alta (codice migliore, ragionamento più solido, italiano più naturale)
  ma più lento se va in offload sulla CPU.
- **Modello più piccolo (3B–4B)** →
  un po’ meno “intelligente”,
  ma molto più veloce perché sta quasi tutto in VRAM.

Quindi sì:

più parametri = più qualità, a costo di velocità (quando la VRAM non basta).

Nel tuo caso, con 4 GB, il punto di equilibrio migliore di solito è tra **3B e 4B**. I 7B valgono la pena solo se la qualità extra ti serve davvero e accetti che vada più lento.

se devo usare gli agenti con i 7b ? che cosa devo fare per ottimizzarlo? timestral può aiutarmi se gli do in pasto il git locale?

Lavorato per 40s >

Due domande separate, ti rispondo chiaramente.

1. Usare agenti con modelli da 7B su 4 GB di VRAM

Sì, si può fare, ma **va ottimizzato bene**, altrimenti diventa lentissimo e instabile.

Cosa fare per ottimizzarlo:

Cosa fare	Perché	Come
Limita il contesto	Il contesto mangia tantissima VRAM	Imposta `num_ctx` a **2048 o massimo 4096**
Offload controllato	Non lasciare che Ollama decida da solo	Usa `OLLAMA_NUM_GPU=20` (o un numero tra 15-25) per tenere più layer possibili sulla GPU
Tieni pochi tool	Ogni tool e ogni risposta aumenta il contesto	Max 3-5 tool semplici (read_file, write_file, run_terminal, git_status)
Cronologia corta	Gli agenti accumulano messaggi velocemente	Cancella o riassumi la history spesso
Modelli bravi in tool calling	Non tutti i 7B sono buoni con gli agenti	Preferisci Qwen2.5 / Qwen3 7B se li hai, o Mistral 7B
Formato di edit semplice	I modelli piccoli sbagliano di più con i diff complessi	Preferisci “riscrivi tutto il file” invece di search/replace sofisticati

Comandi utili:

```bash
# Esempio con contesto limitato
OLLAMA_NUM_CTX=4096 OLLAMA_NUM_GPU=22 ollama run mistral:7b-instruct
```

Realtà pratica:

Su 4 GB i 7B per agenti funzionano, ma sono **lenti** (spesso 5-12 token/s). Per uso quotidiano molti preferiscono restare su **3B–4B** (tipo il tuo `qwen2.5-coder:3b` o `qwen3:4b`) perché sono molto più reattivi e abbastanza bravi per task semplici.

---

2. Timestral e il git locale

Timestral non serve per questo.

Timestral è una famiglia di modelli di **Mistral dedicata alle serie temporali** (previsioni di numeri: vendite, sensori, metriche…). Non è un modello di linguaggio e non capisce codice o git.

Probabilmente intendevi **Mistral** (hai `mistral:7b-instruct`).

Sì, puoi dare in pasto il git locale, ma **non tutto il repository intero**.

Modo corretto:

- Non copiare tutto il codice nel prompt (esplode il contesto).
- Fai usare all'agente dei **tool** per esplorare:
  - `git status`
  - `git log --oneline -10`
  - `git diff`
  - leggere solo i file rilevanti
- Oppure usa un agente che supporta già il workspace (tipo Aider, Continue, o framework locali con tool di file/git).

In pratica: l'agente deve **andare a cercare** le informazioni nel repo, non ricevere tutto in una volta.

Riassunto consigliato per te:

- Per agenti leggeri e veloci → resta su **qwen2.5-coder:3b** o **qwen3:4b**
- Per provare un 7B → usa `mistral:7b-instruct` o un Qwen 7B con contesto basso + offload controllato
- Timestral → no, non è adatto
- Git → sì, ma tramite tool di esplorazione, non dump completo

Vuoi che ti prepari un esempio concreto di configurazione Ollama + agente semplice ottimizzato per 4 GB?

ok ma nel workflow come potrei sfruttarlo "Timestral" o le tipologie simili?

Lavorato per 15s >

Timestral (e e simili) non è un modello di coding, ma un **modello di forecasting su serie temporali**. Serve a prevedere numeri nel tempo (con intervalli di incertezza), non a scrivere o modificare codice.

Può comunque entrare in un workflow di sviluppo o di agenti in modo utile, soprattutto se lavori con sistemi che generano metriche.

Come sfruttarlo concretamente

Caso d'uso	Cosa fa Timestral	Come lo colleghi al resto
Monitoraggio / Observability	Prevede CPU, RAM, latenza, errori, request/s	L'agente LLM riceve l'allarme e va a leggere log, metriche e codice

Caso d’uso	Cosa fa Timestral	Come lo colleghi al resto
Capacity planning	Prevede carico futuro (es. “tra 7 giorni arriveremo a X utenti”)	L’agente propone scaling, ottimizzazioni o alert
Anomaly detection	Segnala quando una metrica esce dalla fascia prevista	L’agente investiga (git log, deploy recenti, errori) e suggerisce fix
Metriche di business	Prevede vendite, conversioni, churn, traffico	Utile se fai prodotti data-driven
Pipeline automatiche	Gira in background su CSV o stream di metriche	Output → tool che chiama l’agente LLM

Workflow tipico “Timestral + agente LLM”

1. **Timestral** gira continuamente (o a intervalli) sulle tue metriche (Prometheus, Influx, CSV esportati, log aggregati…).
2. Quando vede un’anomalia o una previsione critica, genera un evento.
3. L’**agente LLM** (il tuo Qwen/Mistral locale) riceve l’evento e:
   - legge i log recenti
   - controlla `git log` / ultimi deploy
   - analizza il codice correlato
   - propone una diagnosi o un fix
4. Tu (o un altro agente) validi e applichi.

Mistral stessa descrive esattamente questo schema: Timestral fa il pezzo numerico, l’LLM fa il ragionamento e l’azione.

Tipologie simili che puoi considerare

- **Timestral** (Mistral) → zero-shot, probabilistico, gira bene su CPU, piccolo (5M–209M)
- **Chronos** (Amazon)
- **TimesFM** (Google)
- **MOMENT** e altri time-series foundation model

Sono tutti specializzati in numeri nel tempo. Non sostituiscono i modelli di coding che stai usando.

Quando ha senso per te

Ha senso se:

- hai metriche storiche (performance, errori, utilizzo risorse, business)
- vuoi automatizzare “previsione + indagine”
- vuoi allarmi più intelligenti di una soglia fissa

Ha **poco senso** se il tuo workflow è solo “scrivi/modifica codice + git”. In quel caso resta sui modelli di coding (Qwen-Coder, ecc.).

In pratica con il tuo setup

Puoi:

1. Far girare Timestral (o un simile) in un processo separato su CPU (non tocca la VRAM della GPU).
2. Esporre un piccolo tool/API che l'agente LLM può chiamare ("dammi la previsione della metrica X" o "c'è un'anomalia?").
3. Far sì che l'agente usi quel tool insieme a `git`, lettura file e terminal.

Vuoi che ti faccia uno schema concreto di come collegare Timestral (o un'alternativa leggera) a un agente Ollama locale?

> mi servirebbe come poter sfruttare timestral o la tipologia di modello per velocizzarmi ed aiutare il coding agent anche swappando e ricaricando in continuo

Lavorato per 26s >

Sì, si può fare, ma in un modo specifico: Timestral (o modelli simili) non scrive codice, ma può diventare un **supervisore leggero** che aiuta il coding agent a restare veloce e stabile, soprattutto quando fai swapping continuo di modelli.

Idea centrale

Timestral è piccolissimo e gira benissimo **solo su CPU**.

Lo usi per **prevedere** cosa sta succedendo al sistema e all'agente, e in base a quelle previsioni decidi **quando** e **come** fare swap/reload dei modelli di coding.

Come sfruttarlo in pratica

Cosa monitori con Timestral	Cosa può prevedere	Azione automatica sul coding agent
Uso VRAM / RAM nel tempo	Quando stai per andare in offload o OOM	Scarica il modello grande e carica uno più piccolo (es. 3B)
Velocità token/s dell'agente	Quando sta rallentando	Compattta il contesto o fai reset della history
Numero di tool call fallite / retry	Quando l'agente sta "impazzendo"	Ricarica il modello o cambia modello
Lunghezza del contesto	Crescita futura del contesto	Forza riassunto o cancella messaggi vecchi
Carico CPU/GPU complessivo	Momenti di picco	Sospende task pesanti o sposta su modello più leggero

Workflow concreto (swapping continuo)

1. **Timestral** (o Chronos / modello simile) gira in background su CPU e guarda le metriche del sistema + dell'agente.
2. Ogni X secondi (o a ogni turno dell'agente) fa una previsione breve.
3. Un piccolo orchestratore decide:
   - "VRAM sta per saturarsi" → `ollama stop` sul 7B e carica `qwen2.5-coder:3b`

- “Contesto troppo lungo previsto” → riassume la history e ricarica
- “Agente sta fallendo tanti tool” → resetta la sessione o cambia modello

4. Il coding agent continua a lavorare, ma con il modello più adatto al momento.

Vantaggi per te (4 GB VRAM)

- Puoi tenere **sempre** un modello piccolo e veloce pronto.
- Carichi un 7B solo quando serve davvero più qualità.
- Eviti di rimanere bloccato con un modello grande in offload lento.
- Timestral non mangia VRAM → non compete con il coding model.

Schema semplificato

```text
Metriche sistema + agente
        ↓
Timestral (CPU) → previsione
        ↓
Orchestratore (script leggero)
        ↓
Swap / Reload / Context reset del coding agent (Ollama)
```

Cosa ti serve per realizzarlo

- Un piccolo script (Python) che:
  - raccoglie metriche (`nvidia-smi` , `ollama ps` , lunghezza contesto, errori tool…)
  - le passa a Timestral
  - in base alla previsione esegue i comandi di swap (`ollama stop` , `ollama run …`)
- Timestral (o un’alternativa ancora più leggera) sempre in memoria su CPU.

Limitazioni realistiche

- Timestral aiuta a **gestire risorse e stabilità**, non a scrivere codice migliore.
- Lo swapping continuo ha un costo (tempo di caricamento del modello). Conviene farlo solo quando serve davvero.
- Serve un po’ di lavoro di orchestration (non è “plug and play”).

In sintesi:

Timestral (e simili) diventano utili se li usi come **cervello di monitoraggio e previsione**, non come modello di coding. Ti permettono di fare swapping intelligente e di mantenere l’agente veloce anche con poca VRAM.

Vuoi che ti faccia uno scheletro di script (Python) che mostra come collegare metriche → Timestral → comandi di swap su Ollama?

si lo so non voglio che mi genera codice ma a supporto di LLm che generano codice lo so sono 2 cose distinte.

Lavorato per 14s >

Perfetto, siamo allineati.

Timestral (o un modello simile di serie temporali) = **supporto / supervisore**.

LLM di coding (Qwen-Coder, ecc.) = **generatore di codice**.

Sono due ruoli distinti:

- **Timestral** osserva metriche e predice (VRAM, velocità, crescita contesto, fallimenti tool…).
- L'**LLM di coding** scrive e modifica codice.
- Un piccolo orchestratore in mezzo decide quando fare swap, reload o reset in base alle previsioni di Timestral.

Vuoi che ti prepari lo **scheletro concreto** di questo sistema?

Posso darti:

1. Struttura minima (cartelle + responsabilità)
2. Esempio di metriche da raccogliere
3. Come chiamare Timestral (o un'alternativa leggera)
4. Logica di decisione (quando swapare 3B ⇔ 7B, quando resettare il contesto)
5. Comandi Ollama da eseguire

Dimmi solo se preferisci:

- una versione **molto semplice** (script unico), oppure
- una versione un po' più strutturata (facile da estendere).

quindi timestral è il supervisore meno esperto che però gestisce tutto il progetto dietro le quinte

Lavorato per 15s >

Sì, quasi.

Timestral (o un modello simile) è il **supervisore leggero e specializzato** che lavora dietro le quinte. Non è "meno esperto" in senso generico: è **esperto in un compito diverso**.

- L'**LLM di coding** è l'esperto di codice (scrive, modifica, ragiona sul progetto).
- **Timestral** è l'esperto di **numeri nel tempo** (prevede trend di risorse, velocità, stabilità).

Quindi il ruolo corretto è:

> ***Timestral = supervisore di risorse e stabilità***
>
> *(dietro le quinte, senza toccare il codice)*

Gestisce cose come:

- quando la VRAM sta per saturarsi
- quando l'agente sta diventando lento
- quando conviene swappare modello
- quando resettare il contesto

Non gestisce la logica del progetto o le decisioni di coding. Quello resta all'LLM di coding (e all'orchestratore che decide in base alle previsioni di Timestral).

In sintesi:

Ruolo	Chi lo fa	Cosa fa
Esperto di codice	Qwen-Coder / LLM	Scrive e modifica codice
Supervisore	Timestral + script	Monitora e ottimizza risorse
Orchestratore	Tuo piccolo script	Decide quando swapare / resettare

Vuoi che passiamo allo scheletro pratico di questo sistema?

> no voglio capire una cosa invece il Decision model e il timestral ed un LLM posso metterli tutti e 3 insieme per ottimizzare il compito di scrittura codice?

Lavorato per 6s >

Sì, puoi metterli tutti e tre insieme.

È esattamente un'architettura a 3 livelli, dove ognuno ha un ruolo diverso e si aiutano a vicenda per ottimizzare la scrittura di codice.

I tre ruoli

Componente	Ruolo	Cosa fa
LLM di coding (es. Qwen2.5-Coder)	**Esecutore**	Scrive e modifica il codice
Timestral (o simile)	**Supervisore di risorse**	Prevede VRAM, velocità, crescita contesto, stabilità
Decision model	**Cervello strategico**	Decide *cosa* fare e *quando* (quale modello usare, se resettare, se cambiare approccio…)

Come lavorano insieme

```text
Metriche di sistema + stato dell'agente
              ↓
         Timestral
    (prevede problemi di risorse)
              ↓
```

```
   Decision model
(decide la strategia migliore)
          ↓
  LLM di coding
(esegue la scrittura codice)
```

Esempio concreto di flusso:

1. Timestral prevede: “tra poco la VRAM sarà satura e la velocità calerà”.
2. Il Decision model ragiona: “meglio swappare sul 3B, compattare il contesto e continuare con task piccoli”.
3. L’LLM di coding riceve il compito già ottimizzato e scrive il codice nelle condizioni migliori.

Cosa può fare il Decision model

- Scegliere quale modello di coding usare (3B vs 4B vs 7B)
- Decidere se resettare o riassumere il contesto
- Scegliere se fare un task intero o spezzarlo
- Decidere se chiedere conferma o procedere in automatico
- Cambiare strategia se l’agente sta fallendo troppi tool

Importante

- **Timestral** non decide strategia, prevede solo numeri.
- **Decision model** decide la strategia (può essere un LLM piccolo e veloce, tipo 1.5B–3B, o anche regole + un piccolo modello).
- **LLM di coding** esegue.

Puoi anche far sì che il Decision model sia lo stesso LLM di coding in una modalità diversa, ma di solito conviene tenerli separati (uno piccolo e veloce per decidere, uno più capace per scrivere codice).

Risposta breve:

Sì, i tre insieme funzionano molto bene.

Timestral monitora → Decision model decide → LLM di coding scrive.

Vuoi che ti mostri lo schema più chiaro di come collegarli (con i flussi di informazione)?

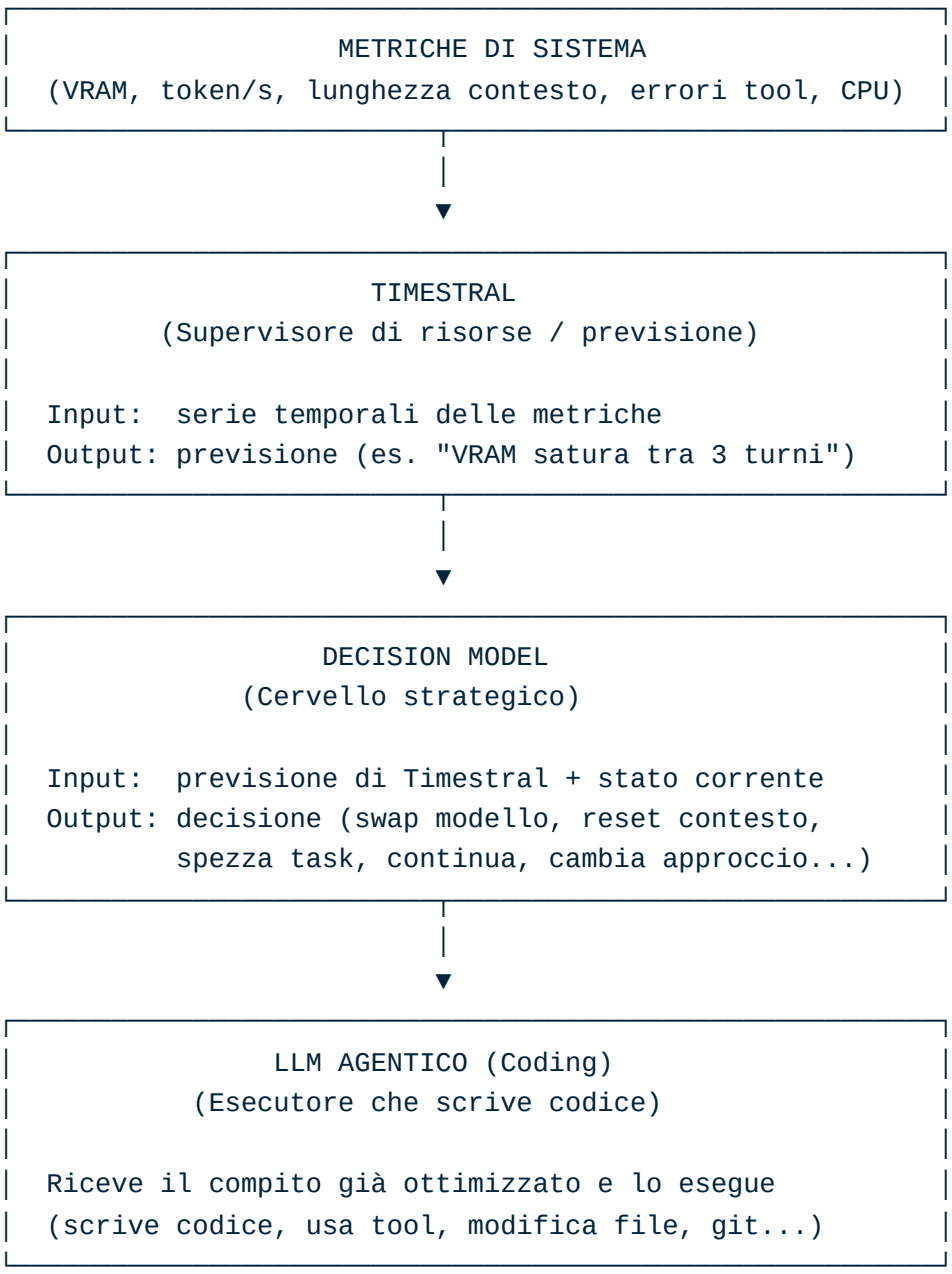
> ok mostrami come sfruttare 1 LLM agentico + Decision model + Timestral tutto insieme

Lavorato per 23s >

Ecco lo schema chiaro e pratico di come far lavorare insieme i **3 componenti**.

Architettura a 3 livelli

text



Ruoli precisi

Componente	Tipo di modello	Compito principale	Quando interviene
Timestral	Time-series (molto piccolo, CPU)	Prevede problemi di risorse	Continuamente / ogni pochi secondi
Decision model	LLM piccolo e veloce (1.5B–3B)	Decide la strategia	Prima di ogni turno importante o quando Timestral segnala qualcosa
LLM agentico	Coding model (3B–7B)	Scrive e modifica codice	Esegue i task

Flusso di un turno tipico

1. **Timestral** guarda le metriche e dice:

   > *"Velocità in calo, contesto in crescita, VRAM al 85%. Tra 2-3 turni rischiamo offload."*

2. **Decision model** riceve questa previsione + lo stato attuale e decide:

   > *“Passa al modello 3B, riassumi la history, spezza il task grande in 2 pezzi più piccoli.”*

3. **LLM agentico** riceve il compito già preparato:

   > *“Lavora solo su questa parte del file, contesto già pulito, modello leggero caricato.”*

4. L’agente scrive il codice nelle condizioni ottimali.

Esempio concreto di decisioni possibili

Il Decision model può scegliere cose del tipo:

- Continua così (tutto ok)
- Swappa su modello più piccolo (3B)
- Swappa su modello più grande (7B) perché serve più qualità
- Resetta / riassumi il contesto
- Spezza il task in sotto-task
- Metti in pausa e chiedi conferma all’utente
- Cambia approccio (es. da “riscrivi tutto” a “modifica mirata”)

Perché funziona bene

- **Timestral** non spreca VRAM (gira su CPU)
- **Decision model** è piccolo e veloce → interviene spesso senza rallentare
- **LLM agentico** lavora solo quando le condizioni sono buone
- Il sistema si adatta in tempo reale invece di andare in offload lento o in errore

Schema riassuntivo in una frase

> ***Timestral prevede → Decision model decide → LLM agentico esegue***

Vuoi che nel prossimo messaggio ti mostri anche:

1. Quali metriche concrete raccogliere, e
2. Un esempio di prompt per il Decision model?

O preferisci prima vedere la struttura a cartelle/script?

ho fatto questo software ma ovviamente è pesantissimo ha mille funzioni. https://github.com/wildlux/Prismalux dovrei allegerirlo e questi 3 metodi aiuterebbe di molto

Lavorato per 40s >

Ho guardato il repo. **Prismalux** è una piattaforma AI locale molto ambiziosa (Qt6/C++, multi-agente, RAG, 50 MCP, GraphMemory, WAN compute, Android, ecc.). Ha senso che sia pesante: ha cresciuto tantissime funzioni.

Risposta diretta

Sì, i 3 metodi (Timestral + Decision model + LLM agentico) possono aiutarti molto, ma in **due modi diversi**:

---

1. Come aiuto a alleggerire il codice (refactoring)

Qui li usi come **strumento di sviluppo**:

Componente	Come ti aiuta sul codice di Prismalux
LLM agentico	Analizza moduli, propone rimozioni, estrae interfacce, spezza file grandi, riscrive parti più semplici
Decision model	Decide *cosa* attaccare prima (es. “questo modulo ha troppe responsabilità, spezzalo”), decide priorità, evita di toccare tutto insieme
Timestral	Monitora durante le sessioni di refactoring: VRAM, velocità, crescita contesto → evita che l’agente vada in offload o perda il filo

Utilità reale:

Ti permette di fare sessioni di refactoring più lunghe e controllate su un codebase grande, senza che l’agente “esploda” di contesto o diventi lentissimo.

Non risolve magicamente la complessità, ma **rende molto più gestibile** il lavoro di alleggerimento.

---

2. Come architettura interna di Prismalux (runtime)

Qui i 3 componenti diventano **parte del software**:

```text
Timestral          → prevede carico risorse (VRAM, token/s, contesto)
Decision model     → decide quale modello caricare, se fare swap, se spezzare task
LLM agentico       → esegue i task di coding / chat / agenti
```

Questo è particolarmente utile su macchine con poca VRAM (come la tua da 4 GB):

Prismalux potrebbe **adattarsi in tempo reale** invece di caricare sempre modelli troppo grandi.

Cosa ti consiglio di fare adesso

1. **Prima priorità: alleggerire il codice**
   Usa il trio come strumento esterno per analizzare e semplificare Prismalux (moduli, dipendenze, responsabilità duplicate).
2. **Seconda priorità: introdurre il trio dentro Prismalux**
   Solo dopo che il codice è più snello, ha senso integrare Timestral + Decision model come sistema di orchestrazione risorse.

Domanda per te

Vuoi che ti aiuti a:

A) Definire un piano di alleggerimento di Prismalux (cosa tenere, cosa isolare, cosa rimuovere/modularizzare),

oppure

B) Progettare come integrare i 3 componenti *dentro* Prismalux come sistema di gestione risorse e agenti?

Dimmi A o B (o entrambi in ordine) e partiamo da lì.

> prismalux ha una gestione di presa parametri di CPU/GPU e temperatura quindi dovrebbe essere facile anche perchè avevo messo un "FRENO a MANO" se le cose si fossero surriscaldate. ma in realtà il mio agente dentro prismalux non è a ciclo continuo perchè mancava l'ottimizzazione di risorse cosa che dovrebbe essere funzionante ora con l'aggiunta di queste 2 oggetti.

Lavorato per 3s >

Esatto, hai già gran parte dell'infrastruttura.

Situazione attuale di Prismalux

- Hai già **monitoraggio** di CPU / GPU / temperatura
- Hai già un **"freno a mano"** di emergenza se si surriscalda
- L'agente **non girava a ciclo continuo** perché mancava l'ottimizzazione intelligente delle risorse

Cosa cambiano i due pezzi nuovi

Pezzo	Cosa aggiunge rispetto a quello che hai già
Timestral	Non si limita a leggere i valori attuali → **prevede** dove andranno (VRAM, temperatura, carico)
Decision model	Non si limita al freno di emergenza → **decide in anticipo** cosa fare (swap modello, ridurre contesto, spezzare task, rallentare, continuare)

Quindi il flusso diventa:

```text
Monitoraggio esistente (CPU/GPU/temp)
        ↓
Timestral (prevede il futuro prossimo)
        ↓
Decision model (sceglie la strategia)
        ↓
Agente può girare in ciclo continuo in sicurezza
```

Il "freno a mano" resta come **ultima rete di sicurezza**.

Timestral + Decision model diventano il **controllo proattivo** che evita di arrivare al freno.

Perché ora l'agente può essere a ciclo continuo

Prima:

- Nessuna previsione → rischiavi di surriscaldare o saturare VRAM
- Solo reazione di emergenza → meglio non lasciare l'agente girare a lungo

Ora:

- Previsione + decisione → l'agente può continuare, ma **adattandosi**
- Quando le risorse peggiorano, riduce carico (modello più piccolo, contesto più corto, pause) invece di bloccarsi

In pratica

Hai già:

1. Sensori (CPU/GPU/temp)
2. Freno di emergenza

Ti manca solo di collegare:
3. **Timestral** → legge i sensori e prevede
4. **Decision model** → decide le azioni di ottimizzazione
5. L'agente → esegue in ciclo, rispettando le decisioni

A quel punto il ciclo continuo diventa realistico, perché non è più "spero che non si surriscaldi", ma "mi adatto prima che si surriscaldi".

Vuoi che ti disegni il flusso preciso di integrazione dentro Prismalux (dove agganciarsi al monitoraggio che hai già e dove far intervenire Decision model)?

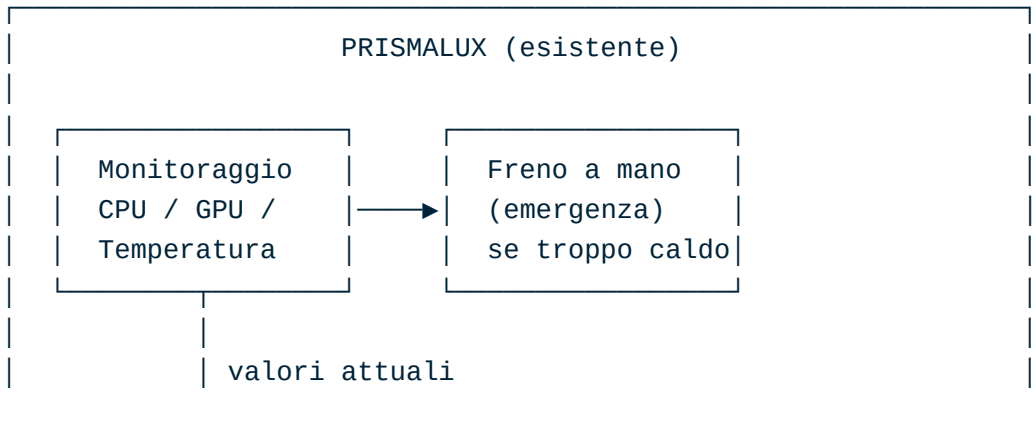
Sì, disegnami il flusso preciso

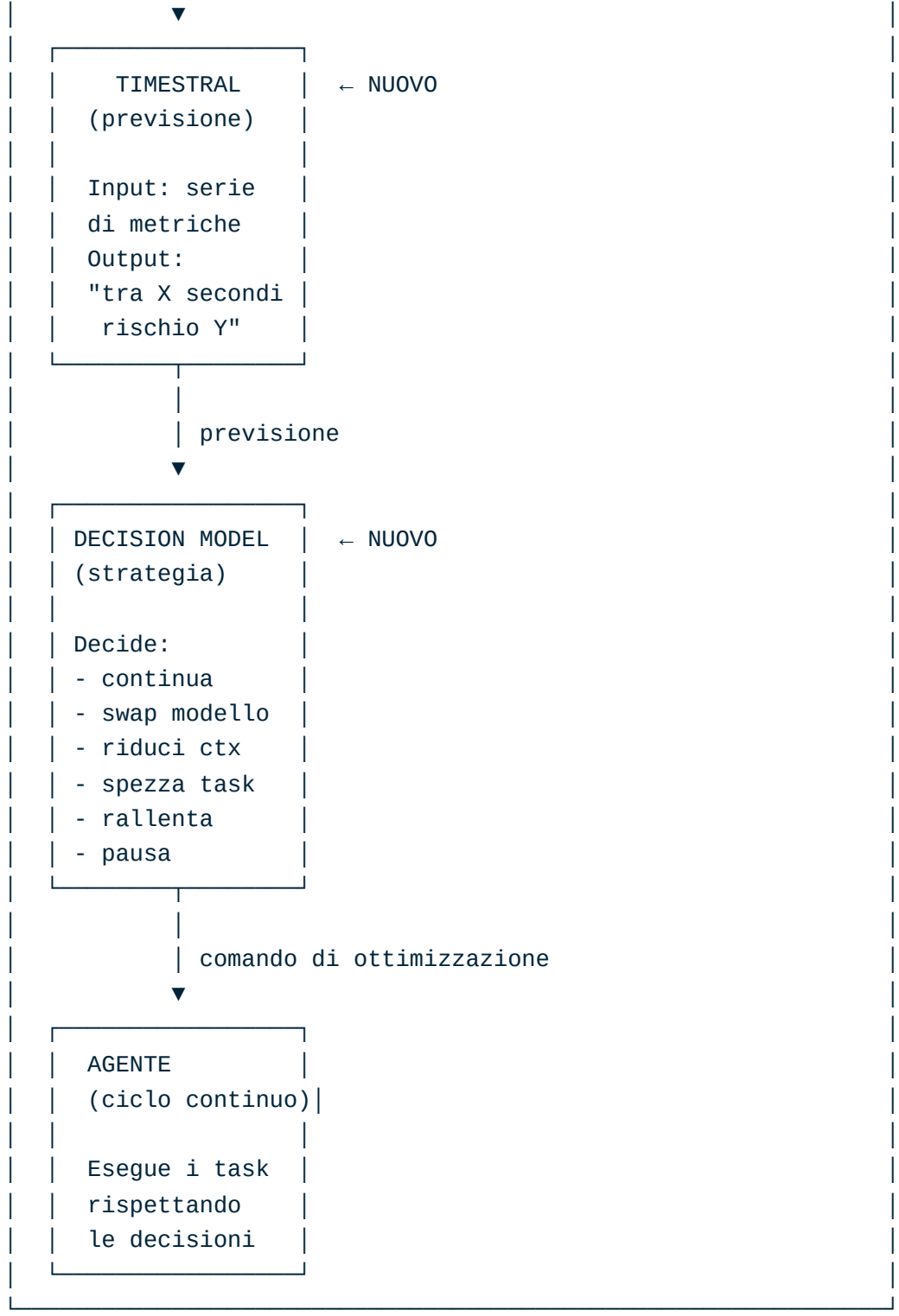
Lavorato per 26s >

Ecco il **flusso preciso** di integrazione dentro Prismalux.

Schema generale

text





Flusso passo-passo (un ciclo)

```text
1. Monitoraggio esistente
   → legge CPU, GPU, VRAM, temperatura, (opzionale: token/s, lunghezza contesto)

2. Timestral
   → riceve la serie storica di queste metriche
   → restituisce una previsione breve
     es: "VRAM al 90% tra ~40 secondi"
         "temperatura in salita, rischio surriscaldamento"
         "velocità token in calo"
```

```
3. Decision model
   → riceve:
        - previsione di Timestral
        - stato attuale dell'agente (task in corso, modello caricato, ctx size)
   → decide UNA azione tra:
        • CONTINUA          (tutto ok)
        • SWAP_MODELLO      (passa a 3B / 4B / 7B)
        • RIDUCI_CONTESTTO  (riassumi o tronca history)
        • SPEZZA_TASK       (dividi il lavoro)
        • RALLENTA          (più pause tra i turni)
        • PAUSA             (aspetta che si raffreddi)
        • FRENO             (chiama il freno a mano esistente)

4. Agente
   → applica la decisione
   → continua il ciclo (o si ferma se è stato attivato il freno)
```

Dove agganciarsi al codice esistente

Parte esistente	Dove collegare
Lettura CPU/GPU/temp	Diventa l'**input** di Timestral (serie temporale)
Freno a mano	Resta l'**ultima rete di sicurezza**. Il Decision model può chiamarlo esplicitamente
Agente / pipeline multi-agente	Riceve i comandi dal Decision model prima di ogni turno (o ogni N turni)
Caricamento modelli (Ollama / llama.cpp)	Controllato dal Decision model (swap)

Ciclo temporale consigliato

```text
Ogni 2–5 secondi:
  Monitoraggio → Timestral → (se serve) Decision model

Prima di ogni turno dell'agente:
  Decision model valuta se cambiare strategia

In caso di emergenza (temp troppo alta):
  Freno a mano (già esistente) ha priorità assoluta
```

Riassunto in una frase

> ***Monitoraggio (già hai) → Timestral prevede → Decision model decide → Agente esegue in ciclo continuo, con il freno a mano come ultima sicurezza.***

Questo è il flusso preciso.

Vuoi che nel prossimo messaggio ti elenchi le **interfacce minime** (funzioni/segnali) da esporre tra questi 4 pezzi?

Sì, elenca le interfacce minime

Lavorato per 20s >

Ecco le **interfacce minime** da esporre tra i 4 pezzi.

1. Monitoraggio (già esistente)

```text
getMetrics() → Metrics
```

Metrics (struttura minima):

```text
cpu_percent: float
gpu_percent: float
vram_used_mb: float
vram_total_mb: float
temperature_c: float
timestamp: int64
```

Opzionale ma utile:

```text
tokens_per_sec: float
context_length: int
model_name: string
```

---

2. Timestral (nuovo)

```text
update(metrics: Metrics)          // riceve un campione
predict() → Prediction            // restituisce la previsione
```

Prediction (struttura minima):

```text
risk_level: enum { OK, WARNING, CRITICAL }
predicted_vram_mb: float
predicted_temp_c: float
```

```
time_to_risk_sec: float          // tra quanti secondi arriva il rischio (0 = già pr
message: string                  // testo breve leggibile
```

---

3. Decision model (nuovo)

```text
decide(prediction: Prediction, agent_state: AgentState) → Decision
```

AgentState (minimo):

```text
current_model: string
context_length: int
task_running: bool
last_action: string
```

Decision (struttura minima):

```text
action: enum {
  CONTINUE,
  SWAP_MODEL,
  REDUCE_CONTEXT,
  SPLIT_TASK,
  SLOW_DOWN,
  PAUSE,
  EMERGENCY_STOP
}
target_model: string             // solo se SWAP_MODEL
reason: string
```

---

4. Agente (esistente, da adattare)

```text
applyDecision(decision: Decision)
getState() → AgentState
```

Comportamento minimo di `applyDecision`:

- `CONTINUE` → non fa nulla
- `SWAP_MODEL` → carica `target_model`

- `REDUCE_CONTEXT` → riassume / tronca history
- `SPLIT_TASK` → spezza il task corrente
- `SLOW_DOWN` → aumenta il delay tra i turni
- `PAUSE` → mette in pausa il ciclo
- `EMERGENCY_STOP` → chiama il freno a mano

---

5. Freno a mano (già esistente)

```text
pullHandbrake(reason: string)
```

Chiamato da:

- Decision model quando `action == EMERGENCY_STOP`
- oppure direttamente dal monitoraggio se temperatura supera una soglia hard

---

Schema di collegamento (chiamate)

```text
loop ogni 2-5 secondi:

  metrics     = Monitoraggio.getMetrics()
  Timestral.update(metrics)
  prediction  = Timestral.predict()

  agent_state = Agente.getState()
  decision    = DecisionModel.decide(prediction, agent_state)

  Agente.applyDecision(decision)

  se decision.action == EMERGENCY_STOP:
      FrenoAMano.pullHandbrake(decision.reason)
```

Riepilogo interfacce (solo i nomi)

Componente	Metodi minimi
Monitoraggio	`getMetrics()`
Timestral	`update(metrics)` , `predict()`
Decision model	`decide(prediction, agent_state)`
Agente	`getState()` , `applyDecision(decision)`

Componente	Metodi minimi
Freno a mano	`pullHandbrake(reason)`

Queste 5 interfacce bastano per far funzionare l'intero sistema.

Vuoi che ti faccia anche la **versione ancora più minimale** (solo i tipi di dato in stile C++/Qt pronti da copiare)?

si ma la versione minimale usa meno risorse no?

Lavorato per 12s >

Sì.

La versione **minimale** usa meno risorse perché:

- meno strutture dati
- meno chiamate
- meno logica di conversione
- meno codice da mantenere e da eseguire a ogni ciclo

Meno pezzi = meno overhead di CPU/RAM e implementazione più leggera.

Versione minimale (solo l'essenziale)

Tipi di dato

C++

```cpp
struct Metrics {
    float cpu;
    float gpu;
    float vram_mb;
    float temp_c;
};

struct Prediction {
    int risk;          // 0=OK, 1=WARNING, 2=CRITICAL
    float time_to_risk; // secondi (0 = già presente)
};

struct Decision {
    int action;        // 0=CONTINUE, 1=SWAP, 2=REDUCE_CTX, 3=PAUSE, 4=STOP
    QString model;     // solo se SWAP
};
```

Interfacce

C++

```
// Monitoraggio
Metrics getMetrics();

// Timestral
void update(const Metrics& m);
Prediction predict();

// Decision model
Decision decide(const Prediction& p, const QString& currentModel, int ctxLen);

// Agente
void applyDecision(const Decision& d);
QString currentModel() const;
int contextLength() const;

// Freno a mano
void pullHandbrake(const QString& reason);
```

Ciclo

C++

```
Metrics m = getMetrics();
update(m);
Prediction p = predict();
Decision d = decide(p, currentModel(), contextLength());
applyDecision(d);

if (d.action == 4)
    pullHandbrake("risk critico");
```

Questa è la forma più leggera possibile che mantiene il sistema funzionante.

Vuoi che ti prepari anche l'ordine di implementazione (da dove partire dentro Prismalux)?

> ok parliamo di modelli va bene timestral e gli altri LLM ok abbiamo deciso invece di Decision model?

Lavorato per 34s >

Ottima domanda. Per il **Decision model** serve qualcosa di diverso dagli altri due.

Cosa deve fare

- Ricevere una previsione corta (da Timestral)
- Guardare lo stato dell'agente (modello attuale, contesto)
- Scegliere **un'azione semplice** tra poche opzioni (CONTINUE / SWAP / REDUCE_CTX / PAUSE / STOP)

Non deve scrivere codice, non deve ragionare a lungo, non deve avere memoria grande.

Deve essere **veloce, piccolo e affidabile**.

Scelte realistiche (con i modelli che hai già)

Opzione	Modello	Dimensione	Perché sì / no
Consigliata	`qwen3:1.7b` o `qwen2.5-coder:1.5b`	~1–1.4 GB	Molto veloce, entra comodo, capisce istruzioni semplici
Buona	`qwen3:4b` / `qwen3.5:4b`	~2.5–3.3 GB	Più intelligente, ma più pesante
Possibile	`phi3:3.8b`	~2.2 GB	Buon ragionamento, un po' più lento
Da evitare	7B	~4+ GB	Troppo pesante per un ruolo di "semaforo"

Raccomandazione

Usa `qwen3:1.7b` (o `qwen2.5-coder:1.5b`) come Decision model.

Motivi:

- gira velocissimo
- lascia quasi tutta la VRAM all'agente di coding
- per decidere tra 5 azioni non serve un modello grande
- puoi tenerlo quasi sempre caricato (o caricarlo solo quando serve)

Schema definitivo dei 3 modelli

Ruolo	Modello	Dove gira
Supervisore risorse	Timestral (o simile)	CPU
Decision model	`qwen3:1.7b` / `qwen2.5-coder:1.5b`	GPU (pochissima VRAM)
Agente coding	`qwen2.5-coder:3b` (o 4b)	GPU

Così hai:

- Timestral che prevede
- Decision model piccolo che decide
- Coding model che esegue

Va bene questa scelta, o preferisci usare lo stesso modello sia come Decision che come coding (con prompt diversi)?

> scusami ma esistono altri decision model perchè usare degli LLM ?
> Ollama logo
> GitHub
> 184.5K
> Download
> Models

Docs
Pricing
profile
Models
Search 260 models
Where
All
260
Local
244
Cloud
21
Capabilities
Tools
104
Thinking
54
Vision
53
Embedding
14
Decision
10
Download size
Any
Up to 8 GB
Up to 16 GB
Up to 32 GB
Up to 64 GB
10 of 260 models
Sort:
Popular
d3
Turn text, structured data, and images into scored choices, yes/no answers, and ratings.
vision
decision
2b
4b
9b
27b
2,967
tev1
A 4B decision model from Together AI for fast classification.
tools
decision
thinking
0.8b
4b

113.1K
nimble
A 9B decision model from Bespoke Labs for fast, typed classification.
tools
decision
thinking
9b
57K
clef-flash
Clef-Flash is a 9B multimodal model that turns a state and a schema of typed questions into decisions.
vision
decision
9b
36.9K
clef
Clef is a 27B multimodal model created by Cloudflare that turns a state and a schema of typed questions into decisions.
Low usage$$$$
|
64K context
|
vision
decision
cloud
27b
36.9K
nimble-v3
A 9B decision model from Bespoke Labs for fast classification, routing, and scoring.
tools
decision
thinking
9b
16.5K
pplx-decider
pplx-decider is a 27B decision model from Perplexity, fine-tuned from Qwen3.8-27B
vision
decision
27b
9,379
kev
Kev scores answers to structured questions about a supplied text state in a single forward pass.
decision
11K
laya
Laya is a 421M decision model from Convai Innovations, fine-tuned from ModernBERT-large.
embedding

decision
14.4K
strands-decider
A 2B decision model from Strands Agents for fast classification, routing, and scoring.
decision
2b
6,402
© 2026 Ollama
Blog
Docs
GitHub
Discord
X (Twitter)
Support
Careers
Privacy
Terms

Lavorato per 28s >

Hai ragione, scusa.

Esistono **modelli specializzati per le decisioni**, e sono molto più adatti di un LLM generico per quel ruolo.

Perché i Decision model dedicati sono meglio

Tipo	Cosa fanno bene	Cosa fanno male
LLM generico (Qwen, Phi…)	Ragionamento aperto, scrittura, chat	Lenti e "chiacchieroni" per scelte semplici
Decision model	Classificazione, routing, scoring, sì/no, scelte strutturate	Non scrivono codice

Il Decision model deve solo scegliere tra poche azioni (`CONTINUE` , `SWAP` , `PAUSE` …).

Per questo un modello **nato per decidere** è più veloce, più stabile e più leggero.

Quelli rilevanti per te (da Ollama)

Hai già installato:

- **`tev1:4b`** ← lo hai in lista

Altri interessanti e leggeri:

Modello	Taglia	Note
tev1	0.8B / 4B	Già ce l'hai (4B). Fatto apposta per decisioni veloci
strands-decider	2B	Piccolo e specializzato in classification/routing
d3	2B / 4B	Scelte, rating, yes/no, anche vision

Modello	Taglia	Note
laya	421M	Piccolissimo (da ModernBERT)

Scelta consigliata per Prismalux

Usa `tev1:4b` (ce l'hai già) oppure, se vuoi ancora più leggero:

- `tev1:0.8b` (se esiste il tag)
- oppure `strands-decider:2b`

Schema aggiornato dei 3 modelli

Ruolo	Modello	Tipo
Supervisore risorse	**Timestral** (o simile)	Time-series
Decision model	`tev1:4b` (o strands-decider / d3:2b)	Decision dedicato
Agente coding	`qwen2.5-coder:3b`	Coding LLM

Molto meglio così: ogni pezzo fa il lavoro per cui è nato.

Vuoi che aggiorni anche le interfacce minime tenendo conto che il Decision model sarà uno di questi (input più strutturato / output più tipizzato)?

> domanda più pragmatica : il ruolo di Supervisore,Decision model ,Agente coding io li ho messi dentro impostazioni e dentro una scheda LLM-DM ma ora abbiamo il tempo timestral cosa mettere ora e se nel futuro escono altre ottimizzazioni che tipologia poso mettere per categorizzarli dentro le impostazioni?

Lavorato per 57s >

Domanda molto pratica. Ti do una **categorizzazione stabile** così puoi organizzare impostazioni e schede senza rifarle ogni volta che esce qualcosa di nuovo.

Categorie consigliate (tipologie)

Organizza per **ruolo funzionale**, non per nome del modello.

Categoria	Ruolo	Esempi attuali / futuri
1. Sensing / Monitoraggio	Legge lo stato del sistema	CPU, GPU, VRAM, temperatura (già ce l'hai)
2. Forecasting / Previsione	Prevede andamento nel tempo	**Timestral**, Chronos, altri time-series
3. Decision / Routing	Sceglie l'azione	**tev1**, strands-decider, d3, nimble
4. Execution / Agente	Esegue il lavoro (coding, tool, chat)	**qwen2.5-coder**, altri coding LLM
5. Memory / Context	Gestisce memoria e contesto	GraphMemory, RAG, riassunto contesto
6. Safety / Guardrail	Limiti e freni	Freno a mano, limiti temp, budget token
7. Orchestration (opzionale)	Coordina i pezzi	pipeline multi-agente, scheduler

Come mapparli adesso in Prismalux

Cosa hai	Categoria
Monitoraggio CPU/GPU/temp	**Sensing**
Timestral	**Forecasting**
tev1 / Decision model	**Decision**
Agente coding	**Execution**
Freno a mano	**Safety**

La scheda **LLM-DM** può restare, ma meglio rinominarla o suddividerla in qualcosa tipo:

- **Risorse & Controllo** → Sensing + Forecasting + Decision + Safety
- **Modelli di esecuzione** → Agente coding (e futuri executor)

Oppure una sola sezione **Ottimizzazione runtime** con sotto-sezioni:

```text
Ottimizzazione runtime
├── Sensing
├── Forecasting     ← Timestral qui
├── Decision        ← tev1 qui
├── Execution       ← coding agent qui
└── Safety          ← freno a mano qui
```

Regola semplice per il futuro

Quando esce una nuova ottimizzazione, chiediti:

> ***Cosa fa?***
>
> - *Legge lo stato?* → ***Sensing***
> - *Prevede il futuro?* → ***Forecasting***
> - *Decide un'azione?* → ***Decision***
> - *Esegue lavoro?* → ***Execution***
> - *Protegge / limita?* → ***Safety***
> - *Ricorda / organizza contesto?* → ***Memory***

Così non crei categorie nuove a ogni modello: li metti nella tipologia giusta.

Risposta diretta sulla tua UI

- **Timestral** → mettilo in **Forecasting** (o "Previsione risorse"), non insieme agli LLM di chat/coding.
- **Decision model** → resta in **Decision**.
- **Agente coding** → resta in **Execution**.
- **Monitoraggio + freno** → **Sensing** + **Safety**.

Se vuoi, nel prossimo messaggio ti propongo i **nomi esatti delle voci** da mettere nel menu Impostazioni (in italiano, pronti da usare).

non mi conviene ancora come nomi...

Lavorato per 19s >

Ok, ripartiamo dai nomi.

Ti propongo **3 alternative più semplici e concrete**. Scegli lo stile che ti convince di più.

Alternativa A — Nomi operativi (cosa fa)

```text
Controllo risorse
├── Sensori
├── Previsione
├── Decisioni
├── Esecuzione
└── Sicurezza
```

- **Sensori** = CPU/GPU/temp
- **Previsione** = Timestral
- **Decisioni** = tev1 / decision model
- **Esecuzione** = agente coding
- **Sicurezza** = freno a mano

Molto chiaro, poco “tecnico”.

Alternativa B — Nomi da “motore”

```text
Motore runtime
├── Lettura
├── Anticipo
├── Scelta
├── Azione
└── Blocco
```

- **Lettura** = sensing
- **Anticipo** = Timestral
- **Scelta** = decision model
- **Azione** = coding agent
- **Blocco** = freno

Più corto, un po' più astratto.

Alternativa C — Nomi da flusso

```text
Ciclo intelligente
├── Misura
├── Prevedi
├── Decidi
├── Esegui
└── Ferma
```

Quasi un verbo per pezzo. Molto intuitivo in UI.

La mia preferenza per Prismalux

Alternativa A:

- Sensori
- Previsione
- Decisioni
- Esecuzione
- Sicurezza

Perché:

- si capisce subito
- resta valida anche tra 2 anni
- non confonde "modello" con "ruolo"

Dimmi quale ti convince di più (A, B, C) oppure dimmi cosa non ti piace dei nomi e ne inventiamo altri.

invece la tipologia tipo robot? sensori controllori attuatori

Lavorato per 15s >

Sì, questa è **molto buona**.

La tipologia da robot/controllo è chiara, standard e resta valida nel tempo.

Mappatura

Tipologia robot	In Prismalux	Cosa ci metti
Sensori	Lettura dello stato	CPU, GPU, VRAM, temperatura, token/s, contesto

Tipologia robot	In Prismalux	Cosa ci metti
Controllori	Chi decide	Timestral (previsione) + Decision model (scelta)
Attuatori	Chi esegue	Agente coding, swap modello, riduci contesto, freno

Versione ancora più precisa (opzionale)

Se vuoi distinguere i due cervelli:

```text
Sensori
Controllori
├── Previsore      ← Timestral
└── Decisore       ← tev1 / decision model
Attuatori
├── Agente coding
└── Sicurezza      ← freno a mano
```

Oppure lasci tutto piatto:

```text
Sensori
Controllori
Attuatori
```

e dentro Controllori metti sia Timestral sia il Decision model.

Perché funziona bene

- Linguaggio già noto (robotica / automazione)
- Non dipende dai nomi dei modelli
- Scalabile: domani aggiungi un nuovo sensore o un nuovo attuatore senza cambiare le categorie
- Si capisce subito il flusso: **Sensori → Controllori → Attuatori**

Consiglio pratico per le impostazioni

Usa esattamente questi tre nomi:

- **Sensori**
- **Controllori**
- **Attuatori**

E nella scheda/descrizione metti una riga:

> *Sensori leggono → Controllori decidono → Attuatori eseguono*

Vuoi che ti prepari anche le sotto-voci minime da mettere dentro ciascuna delle tre?

LLM quindi li rinomino in SCA oppure "Sensori-Controllori-Attuatori.Modelli

Lavorato per 27s >

Meglio **né l'uno né l'altro** così com'è.

Problema

- **SCA** → troppo criptico, tra 3 mesi non ricordi cosa significa
- **Sensori-Controllori-Attuatori.Modelli** → lungo e un po' confuso (sembra un path di file)

Opzioni più pulite

Nome	Pro	Contro
SCA	Corto	Oscuro
Sensori · Controllori · Attuatori	Chiarissimo	Un po' lungo
Controllo runtime	Naturale in impostazioni	Meno "robot"
Architettura SCA	Tiene l'acronimo ma lo spiega	Discreto
Robotica risorse	Creativo	Forse troppo

Consiglio

Nel menu impostazioni usa:

Controllo runtime (SCA)

Dentro la pagina/scheda metti le 3 sezioni:

```text
Sensori
Controllori
Attuatori
```

Così:

- nel menu hai un nome comprensibile
- dentro hai la tipologia robotica chiara
- **SCA** resta come etichetta tecnica, non come unico nome

Se vuoi tenere "Modelli"

Meglio così:

Modelli runtime (SCA)

oppure

Modelli · Sensori / Controllori / Attuatori

Ma la versione più pulita resta:

> ***Controllo runtime (SCA)***
>
> → *Sensori / Controllori / Attuatori*

Vuoi questa, o preferisci un nome ancora più corto nel menu laterale?

se lo chiamo Modelli-AI e poi dentro ci metto la terminologie robotica o per tipologia sarebbe ottimo

Lavorato per 10s >

Sì, è una buona soluzione.

Struttura consigliata

```text
Modelli-AI
├── Sensori
├── Controllori
└── Attuatori
```

Cosa metterci dentro

Sensori

- Monitoraggio CPU / GPU / VRAM / temperatura
- (eventuali altri sensori futuri)

Controllori

- Previsore → Timestral (e simili)
- Decisore → tev1 / decision model

Attuatori

- Agente coding → qwen2.5-coder, ecc.
- Sicurezza → freno a mano
- (swap modello, riduci contesto, ecc.)

Perché funziona

- **Modelli-AI** è chiaro nel menu
- Dentro usi la logica robotica (Sensori → Controllori → Attuatori)
- Resta scalabile: nuovi pezzi vanno nella tipologia giusta
- Non confonde “modello di chat” con “modello di controllo”

Piccolo miglioramento opzionale

Se vuoi essere ancora più chiaro:

```text
Modelli-AI
├── Sensori
├── Controllori
│   ├── Previsore
│   └── Decisore
└── Attuatori
```

Altrimenti tieni le 3 voci piatte: va già benissimo.

Questa organizzazione è solida.

> potrei mettere anche per tipologia cioè se faccio questo usiamo così se faccio matematica usiamo questo oppure se usiamo grafica 3d usiamo ques'tlatro

Lavorato per 3s ›

Sì, puoi farlo, e anzi è utile.

Hai **due assi diversi**:

1. **Ruolo tecnico** → Sensori / Controllori / Attuatori
2. **Tipo di compito** → Coding / Matematica / Grafica 3D / ecc.

Non si escludono: si combinano.

Come organizzarli insieme

```text
Modelli-AI
├── Per ruolo (SCA)
│   ├── Sensori
│   ├── Controllori
│   └── Attuatori
│
└── Per compito
    ├── Coding
    ├── Matematica
    ├── Grafica 3D
    ├── Chat generale
    └── Ricerca / RAG
```

Cosa significa in pratica

Compito	Attuatore tipico	Controllore	Note
Coding	qwen2.5-coder:3b	tev1 + Timestral	priorità velocità + contesto
Matematica	modello forte in reasoning (Phi, Qwen)	stesso controllore	meno tool, più ragionamento
Grafica 3D	modello + tool MCP (Blender, ecc.)	controllore leggero	spesso serve vision o tool specifici
Chat	modello generale	controllore leggero	meno swap aggressivo

Come farlo funzionare

L'utente (o Prismalux) sceglie il **profilo compito**:

- Coding
- Matematica
- Grafica 3D
- …

Quel profilo dice:

- quale **Attuatore** usare di default
- quali parametri (temp, ctx, tool)
- quanto essere aggressivi con swap/previsione

I **Controllori** (Timestral + Decision) restano gli stessi: cambiano solo le regole/priorità in base al profilo.

Consiglio pratico

- **SCA** = architettura interna (sempre presente)
- **Per compito** = profili utente (scelta rapida)

Esempio UI:

```text
Modelli-AI
├── Ruoli (Sensori / Controllori / Attuatori)
└── Profili compito
    ├── Coding
    ├── Matematica
    ├── Grafica 3D
    └── Personalizzato
```

Così hai sia la struttura robotica, sia la comodità "se faccio X uso Y".

Vuoi che ti definisca i **campi minimi di un profilo compito** (cosa salvare per Coding, Matematica, 3D…)?

Crea il profilo compito per coding

Lavorato per 19s >

Profilo compito: Coding

```text
Nome:                Coding
Descrizione:         Scrittura, modifica e debug di codice
Priorità:            Velocità + stabilità risorse + contesto controllato
```

Modelli assegnati (SCA)

Ruolo	Modello	Note
Sensore	Monitoraggio esistente	CPU, GPU, VRAM, temperatura, token/s, ctx
Controllore – Previsore	Timestral (o simile)	Prevede VRAM/temp/carico
Controllore – Decisore	`tev1:4b`	Decisioni rapide (swap, pause, riduci ctx)
Attuatore principale	`qwen2.5-coder:3b`	Default coding
Attuatore leggero	`qwen2.5-coder:1.5b`	Fallback se risorse basse
Attuatore forte (opz.)	`qwen3:4b` / 7B se entra	Solo se Decisore lo permette
Sicurezza	Freno a mano	Emergenza temperatura / criticità

Parametri runtime

```text
Temperatura:          0.2 – 0.4
Context max:          4096 (hard), preferito 2048–3072 su 4 GB
Top-p:                0.9
Max tokens risposta:  1024–2048
Ciclo monitoraggio:   ogni 2–5 secondi
```

Regole del Decisore (Coding)

Condizione	Azione
VRAM / rischio basso	`CONTINUE` con modello attuale
VRAM in salita / rischio medio	`REDUCE_CONTEXT` oppure passa ad Attuatore leggero
Rischio alto (VRAM/temp)	`SWAP` → `qwen2.5-coder:1.5b` o `PAUSE`
Rischio critico	`STOP` → freno a mano

Condizione	Azione
Task lungo + risorse ok	eventuale `SPLIT_TASK`

Comportamento agente

```text
Tool preferiti:       read_file, write_file, terminal, git_status, git_diff
Stile edit:           preferire modifiche mirate; whole-file se file piccolo
History:              corta; riassumere spesso
Ciclo continuo:       sì, governato da Controllori
```

Obiettivi del profilo

1. Restare veloce su 4 GB VRAM
2. Evitare offload lento
3. Permettere ciclo continuo sicuro
4. Usare modello più capace solo quando le risorse lo permettono

---

Valori di default consigliati (riassunto)

```text
attuatore_default     = qwen2.5-coder:3b
attuatore_fallback    = qwen2.5-coder:1.5b
decisore              = tev1:4b
previsore             = Timestral
ctx_preferito         = 2048
ctx_max               = 4096
temperatura           = 0.3
monitor_interval_sec  = 3
```

Questo è il profilo **Coding** pronto da mettere in `Modelli-AI → Profili compito`.

Vuoi adesso anche la versione minimale in formato strutturato (JSON / struct C++/Qt) da salvare nelle impostazioni?